```
import components.map.Map;
import components.map.Map1L;
import components.queue.Queue;
import components.queue.Queue1L;
import components.simplereader.SimpleReader;
import components.simplereader.SimpleReader1L;
import components.simplewriter.SimpleWriter;
import components.simplewriter.SimpleWriter1L;

/**
 * Counts word occurrences in a text file and outputs the results as an HTML
 * table in alphabetical order.
 *
 * @author Zhuangzhuang He
 */
public final class WordCounter {

    /**
     * Characters considered separators between words.
     */
    private static final String SEPARATORS = " \t\n\r,-.!?[]';:/()";

    /**
     * No argument constructor--private to prevent instantiation.
     */
    private WordCounter() {
    }

    /**
     * Reports whether {@code ch} is one of the separator characters.
     *
     * @param ch
     *            character to check
     * @return true iff {@code ch} is a separator
     */
    private static boolean isSeparator(char ch) {
        return SEPARATORS.indexOf(ch) >= 0;
    }

    /**
     * Returns the first word or separator string beginning at {@code position}.
     *
     * @param text
     *            source string
     * @param position
     *            starting index
     * @return first word or separator string at {@code position}
     * @requires 0 <= position < text.length()
     * @ensures nextWordOrSeparator =
     *          text[position, position + |nextWordOrSeparator|)
     */
    private static String nextWordOrSeparator(String text, int position) {
        assert text != null : "Violation of: text is not null";
        assert 0 <= position : "Violation of: 0 <= position";
        assert position < text.length()
                : "Violation of: position < text.length()";

        boolean separator = isSeparator(text.charAt(position));
        int end = position + 1;
        while (end < text.length()
                && isSeparator(text.charAt(end)) == separator) {
            end++;
        }
        return text.substring(position, end);
    }

    /**
     * Escapes special HTML characters in {@code text}.
     *
```

```
     * @param text
     *            text to escape
     * @return escaped text
     */
    private static String escapeHTML(String text) {
        assert text != null : "Violation of: text is not null";

        String result = "";
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch == '&') {
                result += "&amp;";
            } else if (ch == '<') {
                result += "&lt;";
            } else if (ch == '>') {
                result += "&gt;";
            } else if (ch == '"') {
                result += "&quot;";
            } else {
                result += ch;
            }
        }
        return result;
    }

    /**
     * Updates {@code counts} with all words from {@code line}.
     *
     * @param line
     *            line to process
     * @param counts
     *            map of words to their counts
     * @updates counts
     */
    private static void countWordsInLine(String line,
            Map<String, Integer> counts) {
        assert line != null : "Violation of: line is not null";
        assert counts != null : "Violation of: counts is not null";

        int position = 0;
        while (position < line.length()) {
            String token = nextWordOrSeparator(line, position);
            if (!isSeparator(token.charAt(0))) {
                if (counts.hasKey(token)) {
                    int newCount = counts.value(token) + 1;
                    counts.replaceValue(token, newCount);
                } else {
                    counts.add(token, 1);
                }
            }
            position += token.length();
        }
    }

    /**
     * Removes and returns the alphabetically earliest string in {@code words}.
     *
     * @param words
     *            queue of words
     * @return alphabetically earliest word
     * @updates words
     * @requires words /= <>
     */
    private static String removeMin(Queue<String> words) {
        assert words != null : "Violation of: words is not null";
        assert words.length() > 0 : "Violation of: words is not empty";

        String min = words.dequeue();
        int length = words.length();
```

```
        for (int i = 0; i < length; i++) {
            String current = words.dequeue();
            if (current.compareTo(min) < 0) {
                words.enqueue(min);
                min = current;
            } else {
                words.enqueue(current);
            }
        }
        return min;
    }

    /**
     * Sorts {@code words} alphabetically.
     *
     * @param words
     *            queue of words
     * @updates words
     * @ensures words is sorted alphabetically
     */
    private static void sort(Queue<String> words) {
        assert words != null : "Violation of: words is not null";

        Queue<String> sorted = new Queue1L<String>();
        while (words.length() > 0) {
            sorted.enqueue(removeMin(words));
        }
        words.transferFrom(sorted);
    }

    /**
     * Moves all keys from {@code counts} into a sorted queue.
     *
     * @param counts
     *            map of words to counts
     * @return queue containing the words in alphabetical order
     * @updates counts
     */
    private static Queue<String> sortedWords(Map<String, Integer> counts) {
        assert counts != null : "Violation of: counts is not null";

        Queue<String> words = new Queue1L<String>();
        Map<String, Integer> temp = counts.newInstance();
        while (counts.size() > 0) {
            Map.Pair<String, Integer> pair = counts.removeAny();
            words.enqueue(pair.key());
            temp.add(pair.key(), pair.value());
        }
        counts.transferFrom(temp);
        sort(words);
        return words;
    }

    /**
     * Outputs the HTML page.
     *
     * @param inputFileName
     *            input file name
     * @param counts
     *            map of words to counts
     * @param out
     *            output stream
     * @updates out.content
     */
    private static void outputHTML(String inputFileName,
            Map<String, Integer> counts, SimpleWriter out) {
        assert inputFileName != null
                : "Violation of: inputFileName is not null";
        assert counts != null : "Violation of: counts is not null";
```

```
        assert out != null : "Violation of: out is not null";

        Queue<String> words = sortedWords(counts);

        out.println("<html>");
        out.println("<head>");
        out.println("<title>Words Counted in " + escapeHTML(inputFileName)
                + "</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Words Counted in " + escapeHTML(inputFileName)
                + "</h1>");
        out.println("<hr />");
        out.println("<table border=\"1\">");
        out.println("<tr>");
        out.println("<th>Words</th>");
        out.println("<th>Counts</th>");
        out.println("</tr>");

        while (words.length() > 0) {
            String word = words.dequeue();
            out.println("<tr>");
            out.println("<td>" + escapeHTML(word) + "</td>");
            out.println("<td>" + counts.value(word) + "</td>");
            out.println("</tr>");
        }

        out.println("</table>");
        out.println("</body>");
        out.println("</html>");
    }

    /**
     * Main method.
     *
     * @param args
     *            command line arguments
     */
    public static void main(String[] args) {
        SimpleReader in = new SimpleReader1L();
        SimpleWriter out = new SimpleWriter1L();

        out.print("Enter the name of the input file: ");
        String inputFileName = in.nextLine();
        out.print("Enter the name of the output file: ");
        String outputFileName = in.nextLine();

        SimpleReader fileIn = new SimpleReader1L(inputFileName);
        SimpleWriter fileOut = new SimpleWriter1L(outputFileName);

        Map<String, Integer> counts = new Map1L<String, Integer>();
        while (!fileIn.atEOS()) {
            countWordsInLine(fileIn.nextLine(), counts);
        }

        outputHTML(inputFileName, counts, fileOut);

        fileIn.close();
        fileOut.close();
        in.close();
        out.close();
    }
}
```